

Laboratory 6

Expected delivery of lab_06.zip must include:

- Solutions of exercises 1, 3, and 4.
- This file, filled with information and possibly compiled in a pdf format.

This lab will continue to delve into ARM programming.

- 1) Write a program using the ARM assembly that performs the following operations:
 1. Initialize registers $R3$, $R4$, and $R5$ with arbitrary (chosen by you) signed values.
 2. Subtract $R3$ to $R4$ ($R4 - R3$) and store the result in $R6$.
 3. Sum $R4$ to $R5$ ($R4 + R5$) and store the result in $R7$.
- 2) Using the debug log window, change the values of the written program in order to set the following flags to 1, one at a time and when possible:
 - carry
 - overflow
 - negative
 - zero

Report the selected values in the table below:

Updated flag	Hexadecimal representation of the obtained values			
	R4 - R3		R4 + R5	
	R4	R3	R4	R5
Carry = 1	0x0000002A	0x0000000F	0x0000002A	0xFFFFFFFF
Carry = 0	0x0000000A	0x0000000F	0x00000001	0xFFFFFFFF0
Overflow	0x7FFFFFFF	0x80000001	0x7FFFFFFF	0x00000001
Negative	0x0000000A	0x00000064	0x0000000A	0xFFFFFFF9C
Zero	0x000001F4	0x000001F4	0x0000007B	0xFFFFFFF85

Please explain the cases where it is **not** possible to force a **single** FLAG condition:

In the case of the Zero flag ($Z=1$) isolation using subtraction ($R4 - R3$), the attempt fails because $Z=1$ requires the result to be zero, meaning $R4$ must equal $R3$ (as shown by $0x000001F4 - 0x000001F4$). Since $R4$ is greater than or equal to $R3$, the subtraction requires no borrow. In ARM arithmetic, this condition simultaneously sets the Carry flag (C) to 1. Therefore, the operation designed to set $Z=1$ inherently results in $Z=1$ and $C=1$, making isolation impossible for this specific instruction type.

The isolation of the Overflow flag ($V=1$) is unsuccessful because the operation that creates overflow determines the sign of the result. For instance, the values provided for addition ($0x7FFFFFFF + 0x00000001$) cause the result to overflow the positive limit and wrap around to the minimum negative value ($0x80000000$). This immediately sets the Negative flag (N) to 1 because the most significant bit is now set. The result is $V=1$ and $N=1$, preventing the isolation of V . (While the table shows the $R4-R3$ values leading to overflow, these also set $N=1$ for the same reason: $0x7FFFFFFF - 0x80000001$ results in $0xFFFFFFF$, which is negative).

The key failures arise when one arithmetic condition is mathematically forced to trigger another, as seen with the Z and C flags in subtraction.

- 3) Write two versions of a program that performs the following operations:
1. Initialize registers *R1* and *R2* with arbitrary (chosen by you) signed values.
 2. Compare the two registers:
 - If they differ, store in register *R8* the maximum among *R1* and *R2*.
 - Otherwise, perform a logical right shift of 1 on *R1* (is it equivalent to what?), then subtract this value from *R2* and store the result in *R9* (i.e., $R4 = R2 - (R1 \gg 1)$).

The first version should be based on a traditional assembly programming approach using conditional branches, while the second version should be based on a conditional statement execution approach.

Considering a CPU clock frequency (clk) of *16 MHz*, report the number of clock cycles (cc) and the simulation time in milliseconds (ms) in the following table:

	R2 == R3 [cc]	R2 == R3 [ms]	R2 != R3 [cc]	R2 != R3 [ms]
1) Traditional	10	0.00063	12	0.00075
2) Conditional Execution	10	0.00063	10	0.00063

Note: you can change the CPU clock frequency by following the brief guide at the end of the document.

- 4) Write a program that calculates the parity bit of a variable. The parity bit is used for error detection and indicates whether the number of 1s in the binary data is odd (parity = 1) or even (parity = 0), calculated by performing an XOR operation on all bits of the variable: for example, the parity of 0b00000101 is 0 (even parity) because it has two 1s. The variable to be checked is in *R10* that you have to initialize to 0b00110101. After calculating the parity, store the result (0 or 1) in *R13*.

To recap, implement ASM code that does the following:

- Calculate the parity bit of *R10* by XORing all its bits together.
- Store the parity bit result in *R13* (0 for even parity, 1 for odd parity).

- a) Assuming a *15 MHz* clk, report the code size and execution time in the following table:

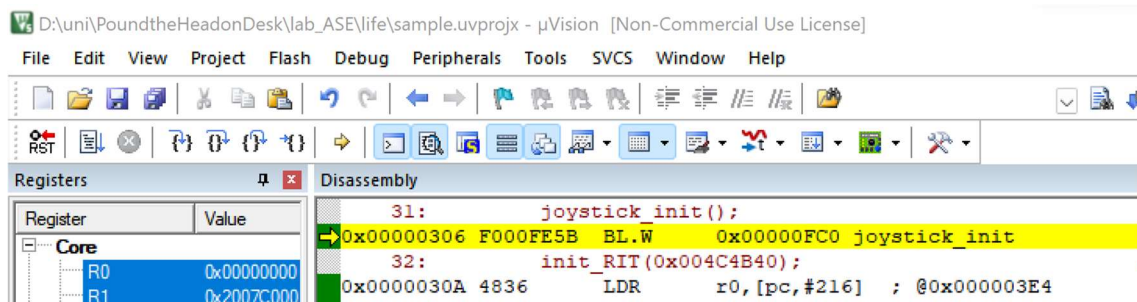
Code size [Bytes]	Execution time
36	0.004 ms

- b) Add some comments regarding the solution you wrote:

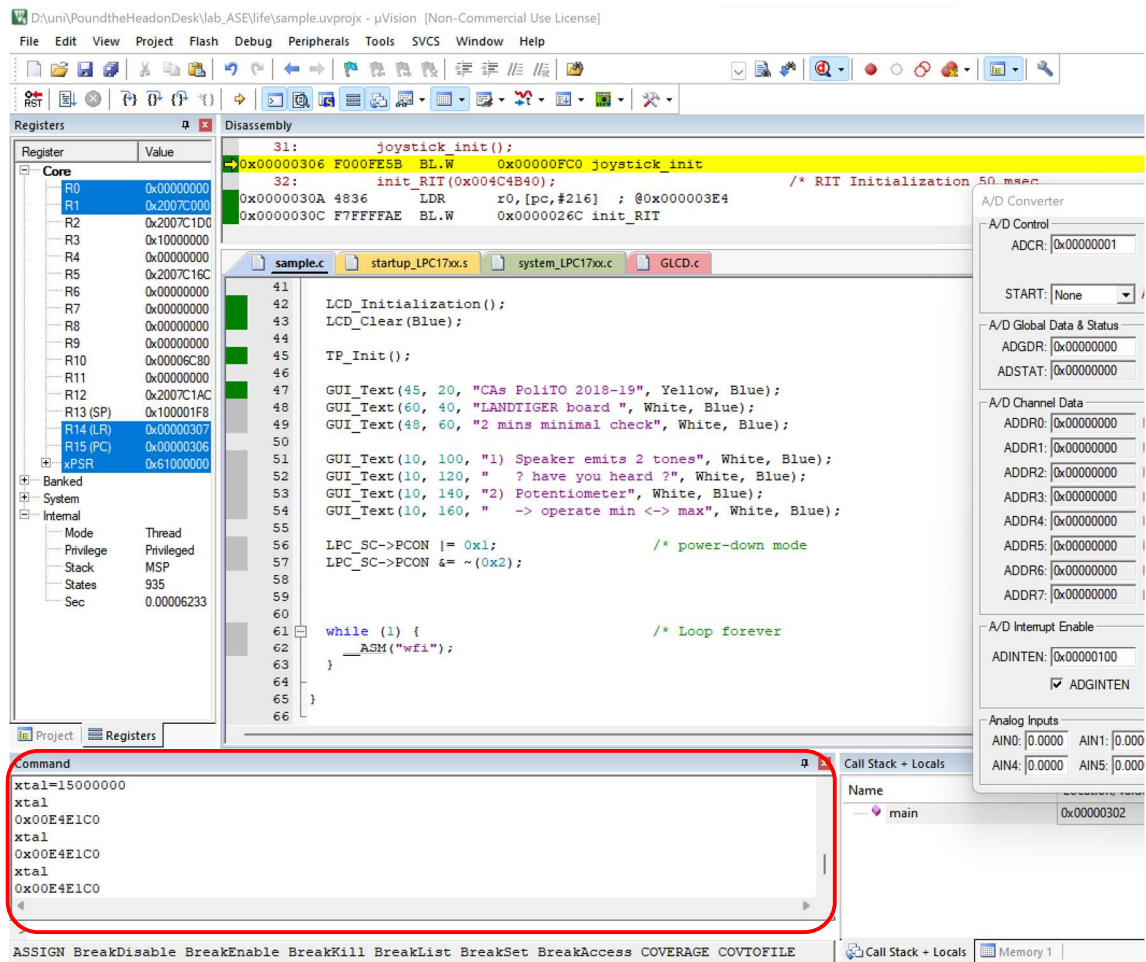
The implemented solution correctly calculates the parity bit by iterating through all 32 bits of register *R10*, using a straightforward approach with XOR operations. The algorithm uses general-purpose registers (<*R10*) to avoid conflicts with reserved registers. A notable concern is the manipulation of *R13*, which serves as the Stack Pointer in ARM Cortex-M3 architecture. While the original SP value is saved in *R1* before modification, it is never restored, leaving the stack in a corrupted state that would prevent any subsequent function calls or interrupt handling. For a production environment, the stack pointer should be immediately restored after debugging, or the result should be stored in a different register.

How to set the CPU clock frequency in Keil

- 1) Launch the debug mode and activate the command console.



- 2) A window will appear:



You can type *xtal* to check its value. To change its value, make a routine assignment, i.e., *xtal=frequency*, keeping in mind that frequency is in Hz must be entered. To set a frequency of 15 MHz, you must write as follows: *xtal=15000000*.